Raspberry Pi Homeserver Administration & Hardening

Complete guide to building and then hardening a Raspberry Pi 4 homeserver — mixing "enterprise-style" services (Samba, Docker, a firewall, an ITSM tool) with a fun personal one (a Minecraft server), and finishing with an access-security pass (non-root containers, SSH key auth, SSH 2FA) once the base install was stable.

> This guide merges what were originally two companion documents — the base administration build and the follow-up hardening pass — into a single, continuous procedure.

0. Prerequisites

- **Hardware:** Raspberry Pi 4 (8 GB recommended)
- **OS:** Debian 13 for Raspberry Pi
- **Connection:** Ethernet (recommended for initial setup)
- **Access:** keyboard/monitor or SSH

1. Network Configuration (Wi-Fi/Ethernet)

Wi-Fi (if needed)

If Wi-Fi isn't working:

```
ip a                    # check the wlan0 interface
dmesg | grep brcm       # check the Broadcom firmware
```

If there's a firmware error:

```
sudo apt update
sudo apt install firmware-brcm80211
sudo reboot
```

Configure Wi-Fi manually:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Add:

```
network={
    ssid="YOUR_WIFI"
    psk="YOUR_PASSWORD"
}
```

Then:

```
sudo systemctl restart dhcpcd
```

2. Full System Update

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

3. Installing and Configuring Samba

3.1 Install Samba

```
sudo apt update
sudo apt install samba
systemctl status smbd   # should show "active (running)"
```

3.2 Create a Shared Folder

```
sudo mkdir -p /srv/partage
sudo chown -R pi:pi /srv/partage
sudo chmod -R 770 /srv/partage
```

(Replace `pi` with your primary user if different.)

3.3 Configure Samba

```
sudo nano /etc/samba/smb.conf
```

Add at the end of the file:

```
[Partage]
  path = /srv/partage
  browseable = yes
  writable = yes
  create mask = 0660
  directory mask = 0770
  valid users = pi
```

3.4 Create a Samba User

```
sudo smbpasswd -a pi   # replace `pi` with your user
sudo smbpasswd -e pi   # enable the user
```

(Use a password different from the system password if you'd like.)

3.5 Restart Samba

```
sudo systemctl restart smbd
```

3.6 Harden Samba

Edit `/etc/samba/smb.conf` and add under `[global]`:

```
interfaces = lo eth0
bind interfaces only = yes
```

Then restart:

```
sudo systemctl restart smbd
```

4. Installing and Configuring Docker

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
sudo apt install docker-compose-plugin
```

5. Firewall Setup (UFW)

```
sudo apt install ufw rsyslog
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw limit ssh
sudo ufw allow 8080/tcp    # GLPI
sudo ufw allow 25565/tcp   # Minecraft (default port)
sudo ufw allow samba       # Samba ports (137/udp, 138/udp, 139/tcp, 445/tcp)
sudo ufw enable
sudo ufw status verbose
```

Expected result:

```
Status: active
Default: deny (incoming), allow (outgoing)

22/tcp                     ALLOW IN    Anywhere
8080/tcp                   ALLOW IN    Anywhere
25565/tcp                  ALLOW IN    Anywhere
137/udp                    ALLOW IN    Anywhere
138/udp                    ALLOW IN    Anywhere
139/tcp                    ALLOW IN    Anywhere
445/tcp                    ALLOW IN    Anywhere
```

Enable logging:

```
sudo ufw logging medium
tail -f /var/log/ufw.log
```

6. SSH Hardening (Baseline)

```
sudo nano /etc/ssh/sshd_config
```

Add:

```
PermitRootLogin no
```

Then:

```
sudo systemctl restart ssh
sshd -T | grep permitrootlogin   # should show "permitrootlogin no"
```

7. Automatic Updates

```
sudo apt install unattended-upgrades apt-listchanges
sudo dpkg-reconfigure unattended-upgrades   # choose YES
systemctl status unattended-upgrades
cat /etc/apt/apt.conf.d/20auto-upgrades
```

Expected result:

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

8. Minecraft Server (Java Edition)

Create a dedicated user:

```
sudo adduser --system --home /opt/minecraft --group minecraft
```

Install Java:

```
sudo apt install openjdk-17-jre-headless
```

Download and launch the server:

```
cd /opt/minecraft
sudo wget https://piston-data.mojang.com/v1/objects/8f05b19832561d8967b870d233b2f3f3234bcc9d/server.jar -O server.jar
sudo chown -R minecraft:minecraft /opt/minecraft
sudo -u minecraft java -Xms1G -Xmx2G -jar server.jar nogui
```

Accept the EULA:

```
sudo nano /opt/minecraft/eula.txt
```

Set:

```
eula=true
```

Then relaunch:

```
sudo -u minecraft java -Xms1G -Xmx2G -jar server.jar nogui
```

Automatic startup (systemd):

```
sudo nano /etc/systemd/system/minecraft.service
```

Add:

```
[Unit]
Description=Minecraft Server
After=network.target

[Service]
User=minecraft
Nice=1
WorkingDirectory=/opt/minecraft
ExecStart=/usr/bin/java -Xms1G -Xmx2G -jar /opt/minecraft/server.jar nogui
Restart=always

[Install]
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reload
sudo systemctl enable minecraft.service
sudo systemctl start minecraft.service
```

9. Installing GLPI with Docker

Create the folders and `docker-compose.yml`:

```
mkdir -p /root/glpi/{config,data,db}
cd /root/glpi
nano docker-compose.yml
```

Add:

```
version: '3.8'

services:
  glpi-db:
    image: mariadb:10.6
    container_name: glpi-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: [set your password here]
      MYSQL_DATABASE: glpi
      MYSQL_USER: glpi
      MYSQL_PASSWORD: [set your password here]
    volumes:
      - ./db:/var/lib/mysql
    networks:
      - glpi-network

  glpi:
    image: glpi/glpi:10.0-nightly
    container_name: glpi
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 1536M
    depends_on:
      - glpi-db
    environment:
      GLPI_DB_HOST: glpi-db
      GLPI_DB_PORT: 3306
      GLPI_DB_NAME: glpi
      GLPI_DB_USER: glpi
      GLPI_DB_PASSWORD: [set your password here]
    volumes:
      - ./config:/etc/glpi
      - ./data:/var/lib/glpi
    ports:
      - "8080:80"
    networks:
      - glpi-network

networks:
  glpi-network:
    driver: bridge
```

Start GLPI:

```
docker compose up -d
```

Check the containers:

```
docker ps
```

Access GLPI:

- URL: `http://[PI_IP]:8080`
- Default credentials: `glpi` / `glpi`

10. Automated Backups

Back up Minecraft:

```
sudo tar -czvf /root/minecraft_backup_$(date +%Y%m%d).tar.gz /opt/minecraft
```

Back up GLPI:

```
sudo tar -czvf /root/glpi_backup_$(date +%Y%m%d).tar.gz /root/glpi
```

Automate with cron:

```
crontab -e
```

Add:

```
0 3 * * * tar -czvf /root/minecraft_backup_$(date +\%Y\%m\%d).tar.gz /opt/minecraft
0 4 * * * tar -czvf /root/glpi_backup_$(date +\%Y\%m\%d).tar.gz /root/glpi
```

Hardening Pass

The base setup above gave the Pi *average* security. The following steps tighten it further **without disrupting** the services already running on the machine — the whole point of hardening a homeserver is strengthening its posture without getting in the way of what it's actually used for day to day.

11. Securing the GLPI Docker Instance

11.1 Create a Dedicated User for GLPI

For better security, run the GLPI container as a dedicated non-root user rather than root.

Create the system user:

```
sudo adduser --system --no-create-home --group glpi-user
```

Fix GLPI folder permissions:

```
sudo chown -R glpi-user:glpi-user /root/glpi
```

Edit `docker-compose.yml`:

Add the `user` directive to the GLPI service in `/root/glpi/docker-compose.yml`:

```
services:
  glpi:
    image: glpi/glpi:10.0-nightly
    user: "1001:1001"  # replace with glpi-user's UID/GID (find via `id glpi-user`)
    ...
```

(Replace `1001:1001` with the UID and GID of `glpi-user`, found via `id glpi-user`.)

Restart the containers:

```
cd /root/glpi
docker compose down
docker compose up -d
```

Verify:

```
docker ps
docker exec -it glpi ps aux   # confirm Apache is running as glpi-user
```

12. SSH Key-Based Login for User `enzo`

1. Generate the key pair (client side)

On your client machine (PC/Mac/Linux), generate an SSH key pair:

```
ssh-keygen -t ed25519 -C "enzo@your-domain.com"
```

Skip the passphrase for unattended use (or set one for extra security).

2. Copy the public key to the Raspberry Pi

```
ssh-copy-id enzo@<PI_IP>
```

(Replace `<PI_IP>` with your Raspberry Pi's IP address.)

3. Configure the SSH server (Raspberry Pi side)

Edit `/etc/ssh/sshd_config`:

```
sudo nano /etc/ssh/sshd_config
```

Modify or add:

```
PubkeyAuthentication yes
PasswordAuthentication no
```

Then restart the SSH service:

```
sudo systemctl restart ssh
```

4. Test the connection

From your client machine:

```
ssh enzo@<PI_IP>
```

If everything is configured correctly, you should log in without a password.

13. Enabling SSH 2FA

1. Install Google Authenticator

```
sudo apt install libpam-google-authenticator
```

2. Configure PAM

Edit `/etc/pam.d/sshd` and add at the top of the file:

```
auth required pam_google_authenticator.so
```

3. Configure SSH

Edit `/etc/ssh/sshd_config` and add/modify:

```
ChallengeResponseAuthentication yes
UsePAM yes
```

4. Initialize 2FA for user `enzo`

Run the following command as `enzo`:

```
google-authenticator
```

- Answer "y" to the prompts to generate backup codes and enable 2FA.
- Scan the QR code with the Google Authenticator app on your smartphone.

5. Restart the SSH service

```
sudo systemctl restart ssh
```

6. Test the connection with 2FA

From your client machine, connect:

```
ssh enzo@<PI_IP>
```

You'll be prompted for the Google Authenticator code after the SSH key.

14. Client-Side Configuration (Optional)

If you're using PuTTY on Windows:

- In PuTTY, go to **Connection > SSH > Auth** and load your private key.
- In **Connection > Data**, enter `enzo` as the username.
- Save the session for quick reconnection.

Conclusion

These steps show how to run a homeserver with services typical of an enterprise environment alongside personal/day-to-day ones — then how to harden it without breaking anything already running on it.